1. Write a program to create a shared library (`.so`) that contains two functions:

- `Addition(int, int)`
- `Substraction(int, int)`

Write a separate client program that loads this library **at runtime** using `dlopen()` and calls both functions using `dlsym()`.

2. Write a C program to demonstrate **runtime dynamic linking** in Linux.

Your program should:

- Load a shared library using `dlopen()`
- Fetch function addresses using `dlsym()`
- Handle errors using `dlerror()`
- Release the library using `dlclose()`

Explain the role of each function used in the program.

3. Create a shared library that contains arithmetic functions:

- Addition
- Substraction
- Multiplication
- Division

Write a client program that dynamically loads the library and calls the required function **based on user choice**.

4. Write a program to demonstrate what happens when:

- A valid shared library is loaded
- An invalid or missing function name is passed to `dlsym()`

Your program should gracefully handle errors using `dlerror()` and print meaningful messages.

5. Explain the difference between **RTLD_LAZY** and **RTLD_NOW** flags used with `dlopen()`.

Write a program that loads the same shared library twice:

- Once using `RTLD_LAZY`
- Once using `RTLD_NOW`

Observe and explain the behavior.

6. Write a program to demonstrate **dynamic library search path resolution** in Linux.

- Place a shared library in a custom directory.
- Write a client program that loads the library without specifying full path.
- Execute the program with and without setting `LD_LIBRARY_PATH`.

Explain the output in both cases.

7. Create a shared library that maintains a **global variable** inside the library.

The library should provide functions to:

- Modify the global variable
- Display its value
- Reset its value

Write a dynamically linked client program to demonstrate persistence of the global variable across multiple function calls.

8. Write a C program to dynamically load a shared library that performs **string operations**:

- Count number of lowercase letters
- Count number of uppercase letters
- Count number of digits

The client program should accept a string from the user and call the appropriate functions using `dlsym()`.

9. Write a program to demonstrate the use of **function pointers with `dlsym()`**.

- Store addresses of multiple dynamically loaded functions in an array of function pointers.
- Invoke functions using this array.
- Display function addresses along with results.

10. Design and implement a **mini dynamic library inspection tool**.

The program should:

- Accept shared library path and symbol name as command-line arguments
- Load the library using `dlopen()`
- Check whether the given symbol exists using `dlsym()`
- Display symbol address if found
- Display appropriate error message otherwise